

INHA UNIVERSITY TASHKENT

SPRING SEMESTER 2024

SOC 2040

SYSTEM PROGRAMMING

CREDITS/HOURS PER WEEK : 3/3

COURSE TYPE : TECHNICAL CORE SEQUENCE

INSTRUCTOR

DR. A. R. NASEER

HEAD & PROFESSOR OF COMPUTER SCIENCE & ENGG

OFFLINE CLASS RULES

TO BE FOLLOWED VERY STRICTLY

- 1. BE PRESENT IN THE CLASS ATLEAST 5 MINUTES BEFORE THE START OF THE CLASSES**
- 2. STUDENTS LATE BY MORE THAN 5 MINUTES WILL NOT BE ALLOWED TO ENTER THE CLASS**
- 3. USE OF CELLPHONES & LAPTOPS STRICTLY PROHIBITED INSIDE THE CLASS**
- 4. CELL PHONES & LAPTOPS SHOULD NOT BE KEPT ON THE DESK**
- 5. CELLPHONES SHOULD BE KEPT EITHER IN YOUR POCKET OR BACKPACK/BAG YOU CARRY**
- 6. SLEEPING IN CLASS IS TOTALLY PROHIBITED**
- 7. BRING A NOTE BOOK FOR THIS COURSE AND MAKE NOTES IN THIS BOOK DURING THE CLASS**
- 8. NO DRINKING & NO EATING IS ALLOWED IN CLASS (PLEASE NOTE THAT CLASSROOM IS NOT CANTEEN)**

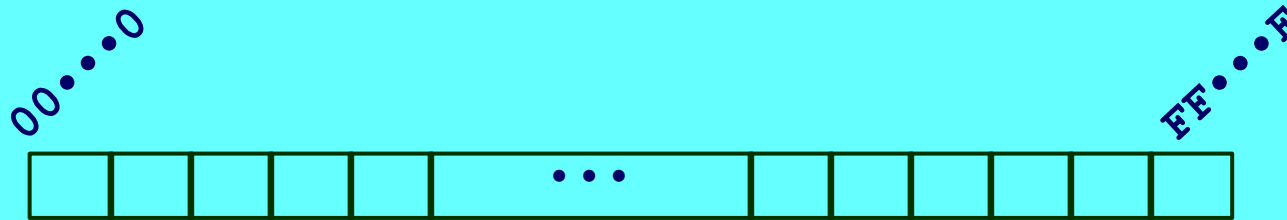
SOC 2040

SYSTEM PROGRAMMING

WEEK 4 LECTURE1

DATA REPRESENTATION

Byte-Oriented Memory Organization



- ★ **Programs refer to data by address**

- Conceptually, envision it as a very large array of bytes
 - ★ In reality, it's not, but can think of it that way
- An address is like an index into that array
 - ★ and, a pointer variable stores an address

- ★ **Note: system provides private address spaces to each “process”**

- Think of a process as a program being executed
- So, a program can clobber its own data, but not that of others

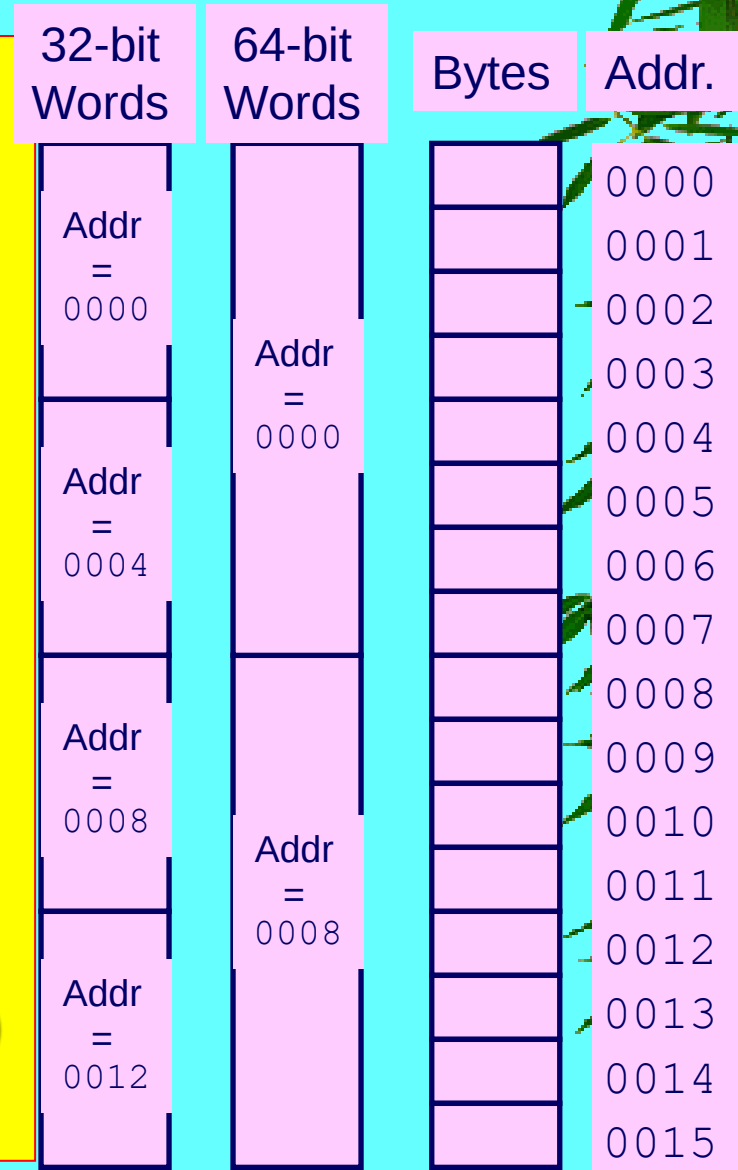
Machine Words

- * Any given computer has a “Word Size”
 - **Nominal size of integer-valued data**
 - * **and of addresses**
 - Until recently, most machines used 32 bits (4 bytes) as word size
 - * **Limits addresses to 4GB (2^{32} bytes)**
 - Increasingly, machines have 64-bit word size
 - * **Potentially, could have 18 EB (exabytes) of addressable memory**
 - * **That's 18.4×10^{18}**
 - **Machines still support multiple data formats**
 - * **Fractions or multiples of word size**
 - * **Always integral number of bytes**

Word-Oriented Memory Organization

★ Addresses Specify Byte Locations

- Address of first byte in word
- Addresses of successive words differ by 2(16-bit), 4 (32-bit) or 8 (64-bit)
- **Convention used in Intel x86-64**
 - byte - 8 bits (1 byte)
 - word - 16 bits (2 bytes)
 - long word – 32 bits (4 bytes)
 - quad word – 64 bits (8 bytes)



Byte Ordering

- ★ So, how are the bytes within a multi-byte word ordered in memory?

Q) How is the word 0x01234567 stored in memory starting from address 0x100

BIG END

LITTLE END

- ★ **Two Conventions**

- **Big Endian**: Sun, PPC Mac, Internet
 - ★ Least significant byte has highest address
- **Little Endian**: x86, ARM processors running Android, iOS, and Windows
 - ★ Least significant byte has lowest address

Byte Ordering Example

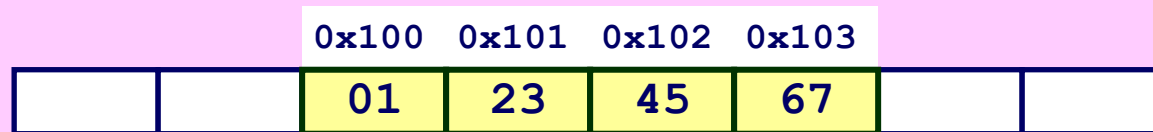
★ Example

- Variable x has 4-byte value of 0x01234567
- Address given by &x is 0x100

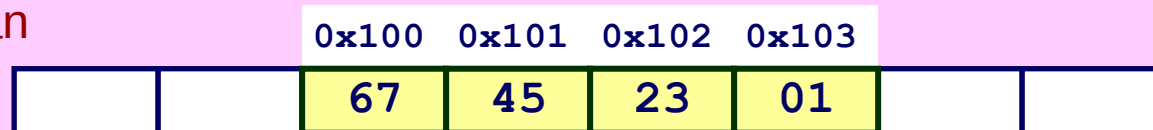
BIG END

LITTLE END

Big Endian



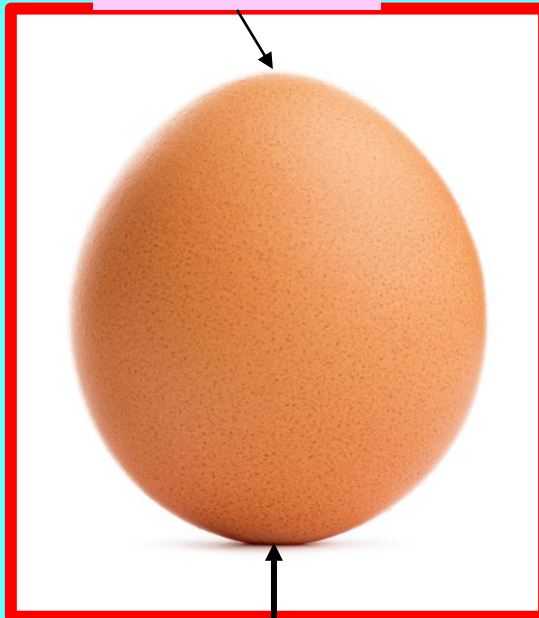
Little Endian



LITTLE-ENDIAN & BIG-ENDIAN - EGG STORY Lilliput Vs Blefuscu

Lilliput and **Blefuscu** are two fictional island nations that appear in the first part of the 1726 novel *Gulliver's Travels* by Jonathan Swift. Lilliput and Blefuscu were intended as, and understood to be, satirical portraits of the Kingdom of Great Britain and the Kingdom of France, respectively, as they were in the early 18th century. The two islands are neighbours in the South Indian Ocean, separated by a channel 800 yards (730 m) wide. Both are inhabited by tiny people who are about one-twelfth the height of ordinary human beings. Both kingdoms are empires, i. e. realms ruled by a self-styled emperor. The capital of Lilliput is Mildendo. In some pictures, the islands are arranged like an egg, as a reference to their egg-dominated histories and cultures.

Little End

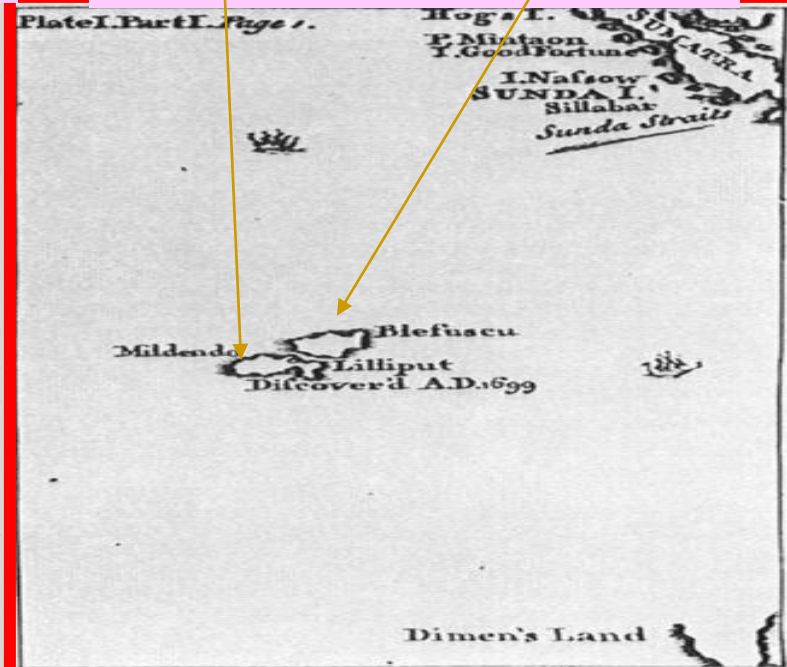


Big End

Gulliver in Lilliput



Lilliput Vs Blefuscu



Representing Integers

Decimal: 15213

Binary: 0011 1011 0110 1101

Hex: 3 B 6 D

Decimal: -15213

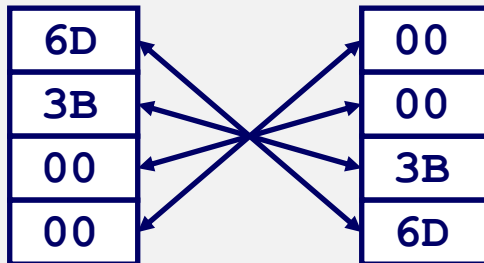
Binary: 1100 0100 1001 0011

Hex: C 4 9 3

int A = 15213;

IA32, x86-64

Sun

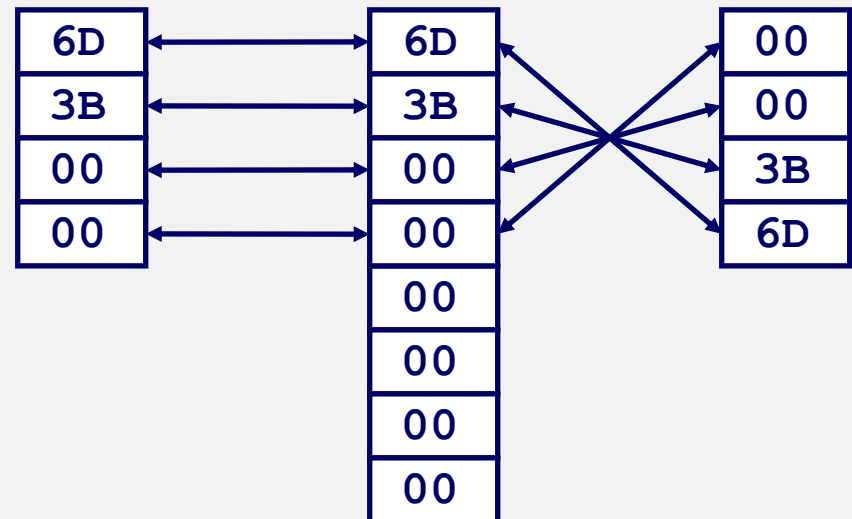


long int C = 15213;

IA32

x86-64

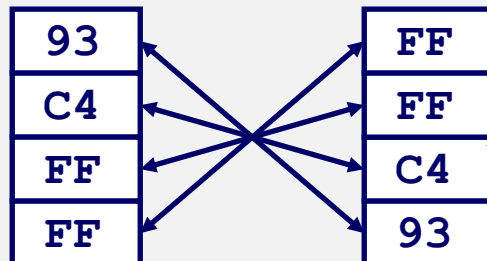
Sun



int B = -15213;

IA32, x86-64

Sun



Two's complement representation

Examining Data Representations

★ Code to Print Byte Representation of Data

- Casting pointer to unsigned char * allows treatment as a byte array

```
#include <stdio.h>
typedef unsigned char *pointer;

void show_bytes(pointer start, size_t len){
    size_t i;
    printf("\n Memory Address    and    Data stored in little endian order\n");
    for (i = 0; i < len; i++)
        printf("%p\t0x%.2x\n", start+i, start[i]);
    printf("\n");
}

int main()
{
    int a = 15213;
    printf("int a = 15213;\n");
    show_bytes((pointer) &a, sizeof(int));
}
```

show_bytes Execution Example

```
int a = 15213;  
printf("int a = 15213;\n");  
show_bytes((pointer) &a, sizeof(int));
```

Result (Linux x86-64):

```
int a = 15213;
```

Memory Address and Data stored in
little endian order

0x7fffb7f71dbc	0x6d
----------------	------

0x7fffb7f71dbd	0x3b
----------------	------

0x7fffb7f71dbe	0x00
----------------	------

0x7fffb7f71dbf	0x00
----------------	------

Representing Pointers

```
int B = -15213;  
int *P = &B;
```

Sun	IA32	x86-64
EF	AC	3C
FF	28	1B
FB	F5	FE
2C	FF	82
		FD
		7F
		00
		00

- ★ Different compilers & machines assign different locations to objects
- ★ Even get different results of pointer addresses each time run program

Representing Strings

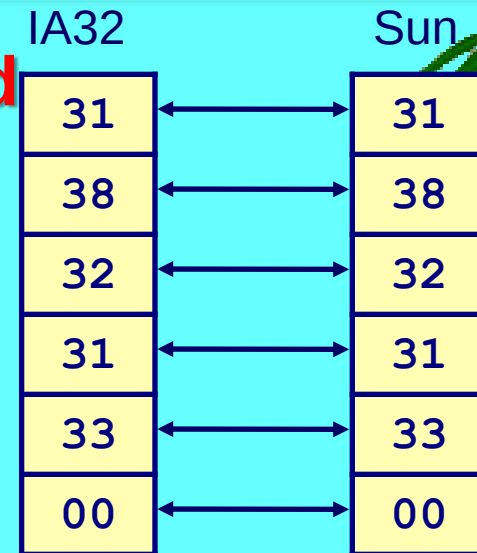
★ Strings in C

- Represented by array of characters
- Each character encoded in ASCII format
 - ★ Standard 7-bit encoding of character set
 - ★ Character “0” has code 0x30
 - Digit i has code $0x30+i$
- String should be null-terminated
 - ★ Final character = 0

```
char S[6] = "18213";
```

★ Compatibility

- Byte ordering not an issue



SOC 2040 SYSTEM PROGRAMMING

Reading Assignments

Chapter 2 of Text Book

➤ Computer Systems : A Programmer's Perspective

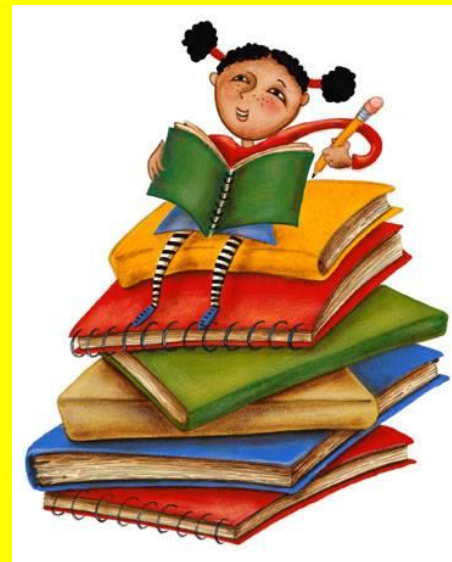
- Randal E. Bryant and David R. O'Hallaron
3rd Edition, Global Edition Prentice Hall 2016
ISBN 10:1-292-10176-8**

TRY OUT ALL PRACTICE PROBLEMS

★ CHAPTER 2 - Practice Problems



- ★ 2.1(page 35 Solution page 134)
- ★ 2.2(page 35 Solution page 134)
- ★ 2.3(page 36 Solution page 135)
- ★ 2.4(page 37 Solution page 135)
- ★ 2.5(page 45 Solution page 135)
- ★ 2.6(page 46 Solution page 136)
- ★ 2.7(page 46 Solution page 136)
- ★ 2.8(page 49 Solution page 136)
- ★ 2.9(page 50 Solution page 136)
- ★ 2.10(page 51 Solution page 137)
- ★ 2.11(page 52 Solution page 137)
- ★ 2.12(page 53 Solution page 137)
- ★ 2.13(page 53 Solution page 138)
- ★ 2.14(page 54 Solution page 138)
- ★ 2.15(page 54 Solution page 138)
- ★ 2.16(page 56 Solution page 139)



SOC 2040

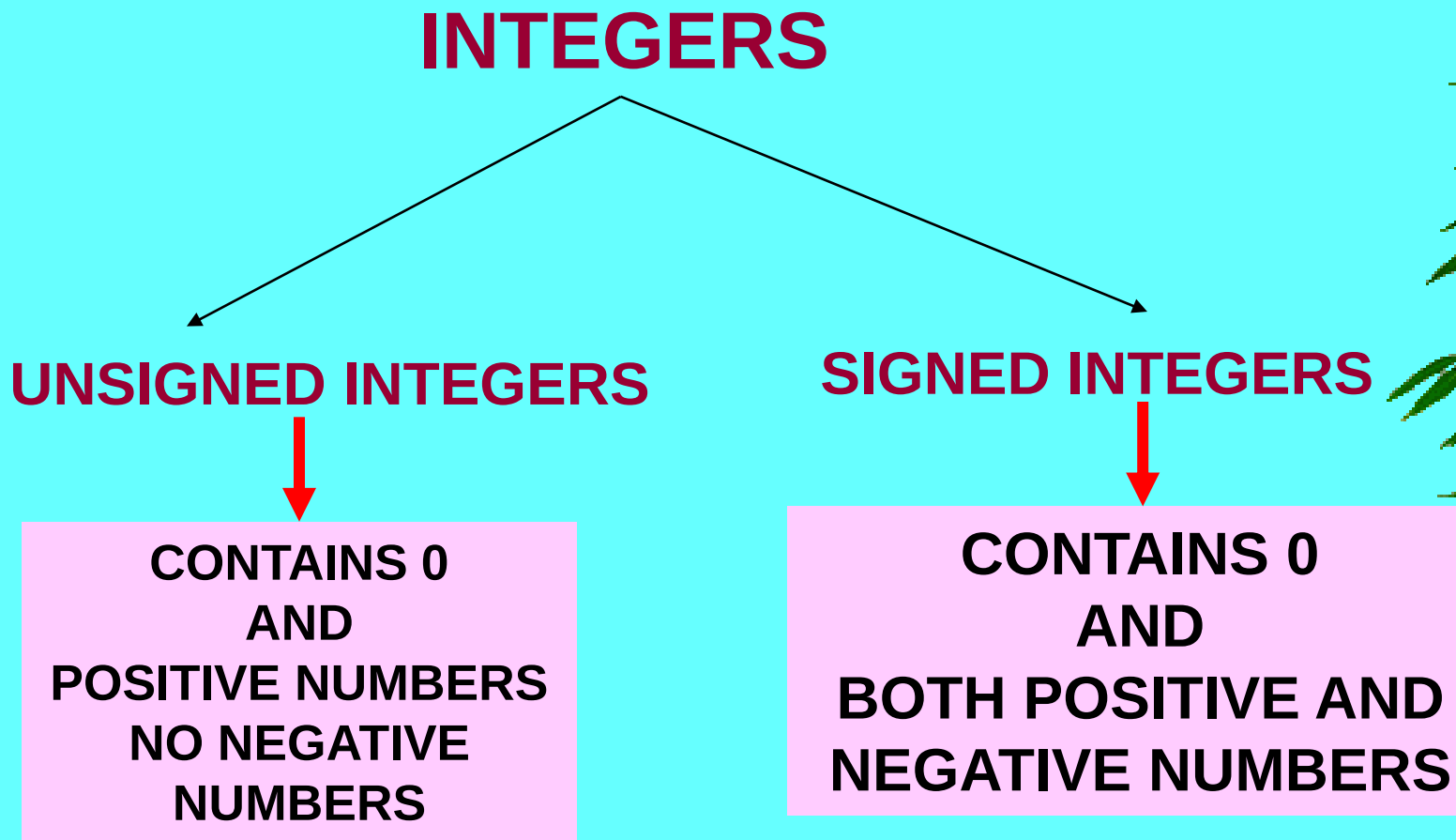
SYSTEM PROGRAMMING

WEEK 4 LECTURE1

INTEGER

REPRESENTATION

Integer Representation



Integer Representation

□ Unsigned Integer

- In general, if an w -bit sequence of binary digits $x_{w-1} x_{w-2} \dots x_1 x_0$ is interpreted as an unsigned integer X and its value is given as

$$X = 2^{w-1} x_{w-1} + 2^{w-2} x_{w-2} + \dots + 2^1 x_1 + 2^0 x_0$$

$$B2U_w(\vec{x}) = X = \sum_{i=0}^{w-1} 2^i x_i$$

$$B2U_w(\vec{x}) \doteq \sum_{i=0}^{w-1} x_i 2^i \quad (2.1)$$

- Use of unsigned integers is insufficient in many cases when positive and negative integers need to be represented.

❑ Unsigned Integer Representation

Given an 8-bit register , the range of numbers that can be stored in 8-bit register is :

255 11111111 ← Maximum Unsigned Integer

254 11111110

.....

.....

1 00000001

0 00000000 ← Minimum Unsigned Integer

Given an 16-bit register , the range of numbers that can be stored in 16-bit register is :

65535 1111111111111111 ← Maximum Unsigned Integer

65534 1111111111111110

.....

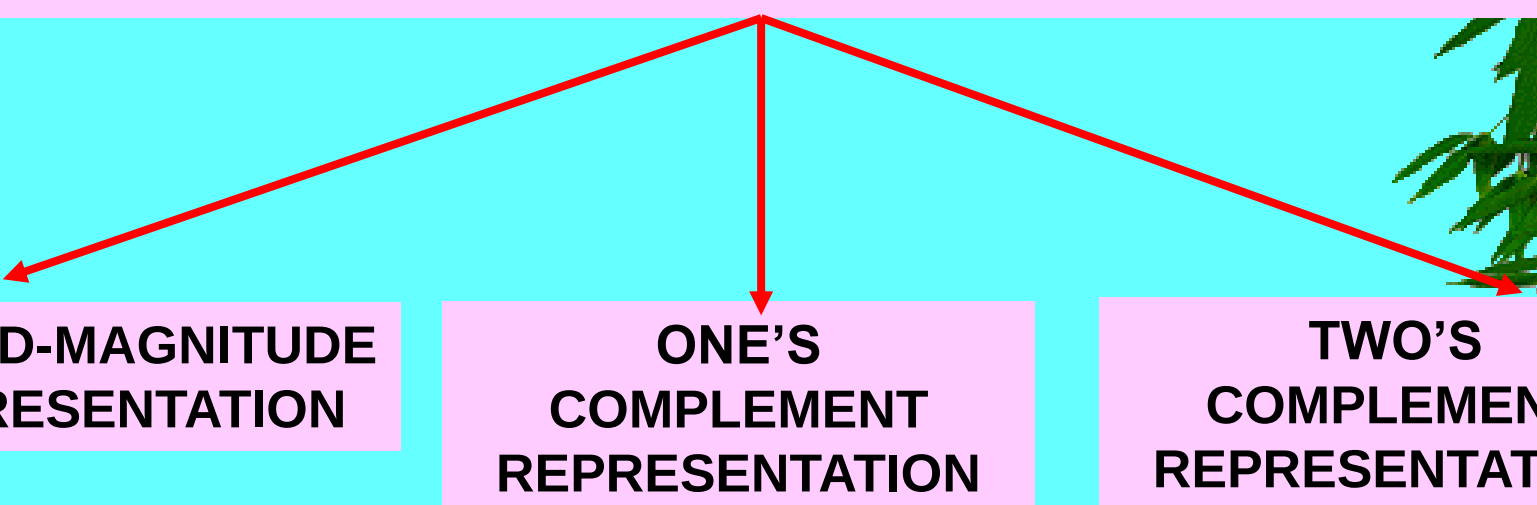
.....

1 0000000000000001

0 0000000000000000 ← Minimum Unsigned Integer

Integer Representation

SIGNED INTEGER REPRESENTATION



```
graph TD; A[SIGNED INTEGER REPRESENTATION] --> B[SIGNED-MAGNITUDE REPRESENTATION]; A --> C[ONE'S COMPLEMENT REPRESENTATION]; A --> D[TWO'S COMPLEMENT REPRESENTATION]
```

**SIGNED-MAGNITUDE
REPRESENTATION**

**ONE'S
COMPLEMENT
REPRESENTATION**

**TWO'S
COMPLEMENT
REPRESENTATION**

Integer Representation

❑ Sign-Magnitude Representation

- Simplest form of representation that employs most significant bit (MSB) of the number as a sign bit.
- If the most significant bit is 0, the number is positive and if the most significant bit is 1, the number is negative
- In an w-bit number, the rightmost w-1 bits hold the magnitude of the integer
- Example, $00010111 = +23$
 $10010111 = -23$

Integer Representation

□ Sign-Magnitude Representation

➤ The general case can be expressed as follows :

$$X = (-1)^{x_{w-1}} \cdot (2^{w-2} x_{w-2} + \dots + 2^1 x_1 + 2^0 x_0)$$

For a w-bit integer X,

$$X = \sum_{i=0}^{w-2} 2^i x_i \quad \text{if } x_{w-1} = 0$$

$$X = - \sum_{i=0}^{w-2} 2^i x_i \quad \text{if } x_{w-1} = 1$$

$$B2S_w(\vec{x}) \doteq (-1)^{x_{w-1}} \cdot \left(\sum_{i=0}^{w-2} x_i 2^i \right)$$

□ Sign-Magnitude Representation

In a Sign-Magnitude scheme, most significant bit (MSB) is used to indicate the sign of the number and hence it is treated as the SIGN BIT :

MSB SIGN BIT = 0 indicates positive number

MSB SIGN BIT = 1 indicates negative number

Given an 8-bit register , the range of numbers that can be stored in 8-bit register is :

+127 01111111

+126 01111110

.....

.....

+1 00000001

+0 00000000

-0 10000000

-1 10000001

.....

.....

-126 11111110

-127 11111111



TWO REPRESENTATIONS FOR 0
+0 & -0

In 8-bit : Positive integer range is + 0 to + 127
Negative integer range is - 0 to - 127

Integer Representation

❑ Sign-Magnitude Representation

❖ Drawbacks

➤ Addition and subtraction require consideration of both the signs of the numbers and their relative magnitudes in order to carry out the required operation.

➤ There are two representations of 0

00000000 = + 0

10000000 = - 0

Integer Representation

❑ One's and Two's Complement of Binary number

❖ One's Complement

In Decimal Number system, 9's complement & 10's Complement of a decimal number are defined

- One's Complement of a binary number can be obtained by subtracting each bit in the binary number from 1 or flip each bit from 1 to 0 or 0 to 1.

9's complement can be computed by subtracting each digit from 9

- For example, one's complement of binary number 11001101 is **00110010**

9's complement of 635 = 364

❖ Two's Complement

10's Complement = 9's Complement + 1
10's complement of 635 = 364 + 1 = 365

- Two's Complement of a binary number can be obtained by first computing one's complement of the given number and then adding one to it, i.e., 2's Complement = 1's Complement + 1.
- For example, 2's complement of binary number 11001101 is **1's complement of 11001101 + 1 = 00110010 + 1 = 00110011**

8 March 2024

One's Complement Integer Representation

In One's Complement Representation of integers, negative integer numbers are represented in One's Complement form and their most significant bit is always 1. There is no change in the representation of positive numbers as their most significant bit is 0

For example :

+10 is 00001010

To represent -10, we take the one's complement of 00001010, that is,

-10 is 11110101

Given an 8-bit register , the range of numbers that can be stored in 8-bit register is :

+127 01111111

+126 01111110

.....

.....

+1 00000001

+0 00000000

- 0 11111111

-1 11111110

.....

.....

-126 10000001

-127 10000000

TWO REPRESENTATIONS FOR 0

+0 & -0



In 8-bit : Positive integer range is + 0 to + 127

Negative integer range is - 0 to - 127

Integer Representation

❑ One's Complement Representation

❖ Given a binary number $x_{w-1} x_{w-2} \dots x_1 x_0$ representing integer in one's complement form and its corresponding value X in decimal can be obtained as follows :

$$X = -x_{w-1} (2^{w-1} - 1) + (2^{w-2} x_{w-2} + \dots + 2^1 x_1 + 2^0 x_0)$$

where Most-significant bit has a value $-(2^{w-1} - 1)$

For example, Given a binary number **11101101** representing integer in one's complement form, determine the corresponding integer value in decimal :

$$X = -x_7 (2^7 - 1) + (2^6 x_6 + 2^5 x_5 + 2^4 x_4 + 2^3 x_3 + 2^2 x_2 + 2^1 x_1 + 2^0 x_0)$$

$$\text{Given } x_7 x_6 x_5 x_4 x_3 x_2 x_1 x_0 = \mathbf{11101101}$$

$$X = -1 * (128 - 1) + (64 * 1 + 32 * 1 + 16 * 0 + 8 * 1 + 4 * 1 + 2 * 0 + 1 * 1)$$

$$= -1 * 127 + (64 + 32 + 0 + 8 + 4 + 0 + 1)$$

$$= -127 + 109 = -18$$

+18 is 00010010
One's complement of
00010010 is
11101101 that is -18

Integer Representation

❑ One's Complement Representation

- Like sign-magnitude scheme, One's complement representation uses the most significant bit as a sign bit, making it easy to test whether an integer is positive or negative.
- It differs from sign-magnitude representation in the way that the other bits are interpreted.
- Here Most-significant bit has a value $-(2^{w-1} - 1)$ rather than -2^{w-1}

$$X = -x_{w-1}(2^{w-1} - 1) + (2^{w-2}x_{w-2} + \dots + 2^1x_1 + 2^0x_0)$$

$$B2O_w(\vec{x}) \doteq -x_{w-1}(2^{w-1} - 1) + \sum_{i=0}^{w-2} x_i 2^i$$

- One's Complement of 01100010 is 10011101
- There are two representations of 0

$$00000000 = +0$$

$$11111111 = -0$$

TWO's Complement Representation

In Two's Complement Representation of integers, negative integer numbers are represented in Two's Complement form and their most significant bit is always 1. There is no change in the representation of positive numbers as their most significant bit is 0

For example :

+10 is 00001010

To represent -10, we take the two's complement of 00001010, that is,

-10 is 11110110

Given an 8-bit register , the range of numbers that can be stored in 8-bit register is :

+127 01111111

+126 01111110

.....

.....

+1 00000001

0 00000000

-1 11111111

.....

.....

-126 10000010

-127 10000001

-128 10000000

ONLY ONE REPRESENTATION FOR 0

In 8-bit : Positive integer range is 0 to + 127
Negative integer range is - 1 to - 128

In 16-bit : Positive integer range is 0 to + 32767
Negative integer range is - 1 to - 32768

Integer Representation

❑ Two's Complement Representation

❖ Given a binary number $x_{w-1} x_{w-2} \dots x_1 x_0$ representing integer in Two's complement form and its corresponding value X in decimal can be obtained as follows :

$$X = -x_{w-1} (2^{w-1}) + (2^{w-2} x_{w-2} + \dots + 2^1 x_1 + 2^0 x_0)$$

where Most-significant bit has a value $-(2^{w-1})$

For example, Given a binary number **11101101** representing integer in Two's complement form, determine the corresponding integer value in decimal :

$$X = -x_7 (2^7) + (2^6 x_6 + 2^5 x_5 + 2^4 x_4 + 2^3 x_3 + 2^2 x_2 + 2^1 x_1 + 2^0 x_0)$$

$$\text{Given } x_7 x_6 x_5 x_4 x_3 x_2 x_1 x_0 = \mathbf{11101101}$$

$$X = -1 * (128) + (64 * 1 + 32 * 1 + 16 * 0 + 8 * 1 + 4 * 1 + 2 * 0 + 1 * 1)$$

$$= -1 * 128 + (64 + 32 + 0 + 8 + 4 + 0 + 1)$$

$$= -128 + 109 = -19$$

+19 is 00010011

2's complement of
00010011 is

11101100 +1 that is -
11101101 = -19

Integer Representation

❑ Two's Complement Representation

- ★ The general expression defining the two's complement representation for both positive and negative numbers is given as

$$X = -x_{w-1} 2^{w-1} + (2^{w-2} x_{w-2} + \dots + 2^1 x_1 + 2^0 x_0)$$

For a w-bit integer X,

$$X = -2^{w-1} x_{w-1} + \sum_{i=0}^{w-2} 2^i x_i$$

$$B2T_w(\vec{x}) \doteq -x_{w-1} 2^{w-1} + \sum_{i=0}^{w-2} x_i 2^i \quad (2.3)$$

Q) Given a binary number X= 11011001, write the corresponding decimal value in the following Representations of the binary number X :

- (i) Unsigned form (B2U),**
- (ii) Signed Magnitude form (B2S) ,**
- (iii) One's Complement form (B2O),**
- (iv) Two's Complement form (B2T)**

$$\begin{aligned} \text{(i) B2U(11011001)} &= 1 \times 2^7 + 1 \times 2^6 + 0 \times 2^5 + 1 \times 2^4 + 1 \times 2^3 + 0 \times 2^2 + 0 \times 2^1 + 1 \times 2^0 \\ &= 128 + 64 + 0 + 16 + 8 + 0 + 0 + 1 = \mathbf{217} \end{aligned}$$

$$\begin{aligned} \text{(ii) B2S(11011001)} &= - (1 \times 2^6 + 0 \times 2^5 + 1 \times 2^4 + 1 \times 2^3 + 0 \times 2^2 + 0 \times 2^1 + 1 \times 2^0) \\ &= - (64 + 0 + 16 + 8 + 0 + 0 + 1) = \mathbf{- 89} \end{aligned}$$

$$\begin{aligned} \text{(iii) B2O(11011001)} &= -1 \times (2^7 - 1) + 1 \times 2^6 + 0 \times 2^5 + 1 \times 2^4 + 1 \times 2^3 + 0 \times 2^2 + 0 \times 2^1 + 1 \times 2^0 \\ &= -127 + 64 + 0 + 16 + 8 + 0 + 0 + 1 = \mathbf{-38} \end{aligned}$$

$$\begin{aligned} \text{(iv) B2T(11011001)} &= -1 \times 2^7 + 1 \times 2^6 + 0 \times 2^5 + 1 \times 2^4 + 1 \times 2^3 + 0 \times 2^2 + 0 \times 2^1 + 1 \times 2^0 \\ &= -128 + 64 + 0 + 16 + 8 + 0 + 0 + 1 = \mathbf{-39} \end{aligned}$$

Q) Given a binary number X= 110011001100, write the corresponding decimal value in the following Representations of the binary number X :

- (i) Unsigned form (B2U),**
- (ii) Signed Magnitude form (B2S) ,**
- (iii) One's Complement form (B2O),**
- (iv) Two's Complement form (B2T)**

(i)B2U(110011001100)

$$\begin{aligned} &= 1 \times 2^{11} + 1 \times 2^{10} + 0 \times 2^9 + 0 \times 2^8 + 1 \times 2^7 + 1 \times 2^6 + 0 \times 2^5 + 0 \times 2^4 + 1 \times 2^3 + 1 \times 2^2 + 0 \times 2^1 + 0 \times 2^0 \\ &= 2048 + 1024 + 0 + 0 + 128 + 64 + 0 + 0 + 8 + 4 + 0 + 0 = 3276 \end{aligned}$$

(ii)B2S(110011001100)

$$\begin{aligned} &= - (1 \times 2^{10} + 0 \times 2^9 + 0 \times 2^8 + 1 \times 2^7 + 1 \times 2^6 + 0 \times 2^5 + 0 \times 2^4 + 1 \times 2^3 + 1 \times 2^2 + 0 \times 2^1 + 0 \times 2^0) \\ &= - (1024 + 0 + 0 + 128 + 64 + 0 + 0 + 8 + 4 + 0 + 0) = -1228 \end{aligned}$$

(iii)B2O(110011001100)

$$\begin{aligned} &= -1 \times (2^{11} - 1) + (1 \times 2^{10} + 0 \times 2^9 + 0 \times 2^8 + 1 \times 2^7 + 1 \times 2^6 + 0 \times 2^5 + 0 \times 2^4 + 1 \times 2^3 + 1 \times 2^2 + 0 \times 2^1 + 0 \times 2^0) \\ &= -2047 + (1024 + 0 + 0 + 128 + 64 + 0 + 0 + 8 + 4 + 0 + 0) = -819 \end{aligned}$$

(iv)B2T(110011001100)

$$\begin{aligned} &= -1 \times (2^{11}) + (1 \times 2^{10} + 0 \times 2^9 + 0 \times 2^8 + 1 \times 2^7 + 1 \times 2^6 + 0 \times 2^5 + 0 \times 2^4 + 1 \times 2^3 + 1 \times 2^2 + 0 \times 2^1 + 0 \times 2^0) \\ &= -2048 + (1024 + 0 + 0 + 128 + 64 + 0 + 0 + 8 + 4 + 0 + 0) = -820 \end{aligned}$$

Encoding Integers

Unsigned

$$B2U(X) = \sum_{i=0}^{w-1} x_i \cdot 2^i$$

Two's Complement

$$B2T(X) = -x_{w-1} \cdot 2^{w-1} + \sum_{i=0}^{w-2} x_i \cdot 2^i$$

```
short int x = 15213;  
short int y = -15213;
```

Sign
Bit

- ★ C short 2 bytes long

	Decimal	Hex	Binary
x	15213	3B 6D	00111011 01101101
y	-15213	C4 93	11000100 10010011

- ★ Sign Bit
 - For 2's complement, most significant bit indicates sign
 - ★ 0 for nonnegative
 - ★ 1 for negative

Two-complement Encoding Example

x =	15213: 00111011 01101101			
y =	-15213: 11000100 10010011			
Weight	15213		-15213	
1	1	1	1	1
2	0	0	1	2
4	1	4	0	0
8	1	8	0	0
16	0	0	1	16
32	1	32	0	0
64	1	64	0	0
128	0	0	1	128
256	1	256	0	0
512	1	512	0	0
1024	0	0	1	1024
2048	1	2048	0	0
4096	1	4096	0	0
8192	1	8192	0	0
16384	0	0	1	16384
-32768	0	0	1	-32768
Sum	15213		-15213	

Numeric Ranges

* Unsigned Values

- $UMin = 0$
000...0
- $UMax = 2^w - 1$
111...1

* Two's Complement Values

- $TMin = -2^{w-1}$
100...0
- $TMax = 2^{w-1} - 1$
011...1

* Other Values

- Minus 1
111...1

Values for $w = 16$ bits

	Decimal	Hex	Binary
UMax	65535	FF FF	11111111 11111111
TMax	32767	7F FF	01111111 11111111
TMin	-32768	80 00	10000000 00000000
-1	-1	FF FF	11111111 11111111
0	0	00 00	00000000 00000000

Values for Different Word Sizes

	W			
	8	16	32	64
UMax	255	65,535	4,294,967,295	18,446,744,073,709,551,615
TMax	127	32,767	2,147,483,647	9,223,372,036,854,775,807
TMin	-128	-32,768	-2,147,483,648	-9,223,372,036,854,775,808

★ Observations

- $|TMin| = TMax + 1$
- $UMax = 2 * TMax + 1$
(Asymmetric Range)

■ C Programming

- `#include <limits.h>`
- Declares constants, e.g.,
 - `ULONG_MAX`
 - `LONG_MAX`
 - `LONG_MIN`
- Values platform specific

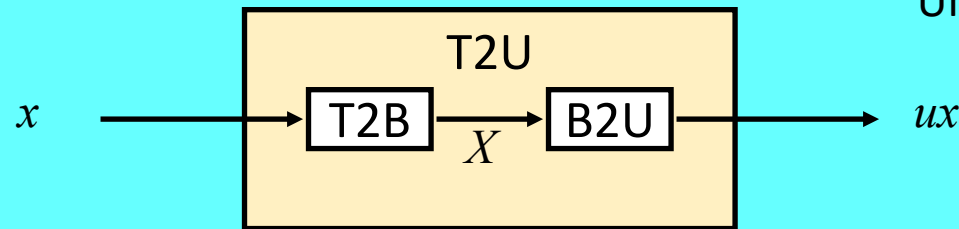
Unsigned & Signed Numeric Values

X	B2U(X)	B2T(X)
0000	0	0
0001	1	1
0010	2	2
0011	3	3
0100	4	4
0101	5	5
0110	6	6
0111	7	7
1000	8	-8
1001	9	-7
1010	10	-6
1011	11	-5
1100	12	-4
1101	13	-3
1110	14	-2
1111	15	-1

- * Equivalence
 - Same encodings for nonnegative values
- * Uniqueness
 - Every bit pattern represents unique integer value
 - Each representable integer has unique bit encoding
- * $\Rightarrow$ Can Invert Mappings
 - $U2B(x) = B2U^{-1}(x)$
 - * Bit pattern for unsigned integer
 - $T2B(x) = B2T^{-1}(x)$
 - * Bit pattern for two's comp integer

Mapping Between Signed & Unsigned

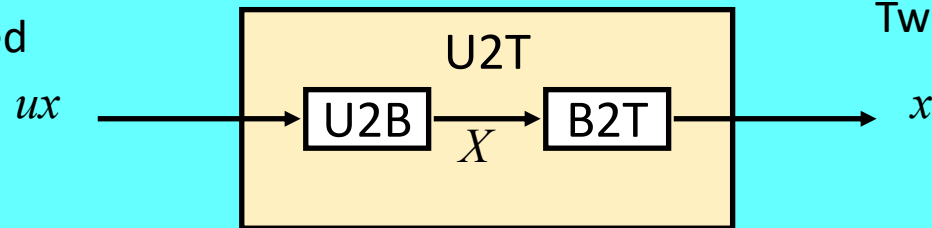
Two's Complement



Unsigned

Maintain Same Bit Pattern

Unsigned



Two's Complement

Maintain Same Bit Pattern

- ★ Mappings between unsigned and two's complement numbers:

Keep bit representations and reinterpret

Mapping Signed $\leftrightarrow$ Unsigned

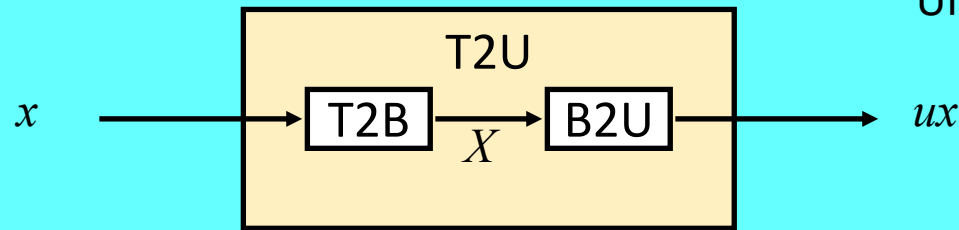
Bits	Signed		Unsigned
0000	0	$\longleftrightarrow$ =	0
0001	1		1
0010	2		2
0011	3		3
0100	4		4
0101	5		5
0110	6		6
0111	7		7
1000	-8	$+ 2^4 = + 16$ $\longrightarrow$	8
1001	-7		9
1010	-6	$- 2^4 = - 16$ $\longleftarrow$	10
1011	-5		11
1100	-4	$+/- 16$ $\longleftrightarrow$	12
1101	-3		13
1110	-2		14
1111	-1		15

w=4

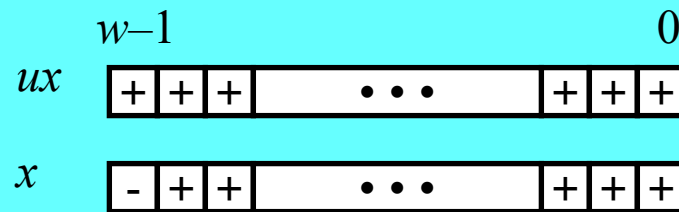
Relation between Signed & Unsigned

Two's Complement

Unsigned



Maintain Same Bit Pattern



Large negative weight
becomes
Large positive weight

$$B2U_w(T2B_w(x)) = T2U_w(x) = x_{w-1}2^w + x \quad (2.5)$$

$$T2U_w(x) = \begin{cases} x + 2^w, & x < 0 \\ x, & x \geq 0 \end{cases} \quad (2.6)$$

$$B2T_w(U2B_w(u)) = U2T_w(u) = -u_{w-1}2^w + u \quad (2.7)$$

$$U2T_w(u) = \begin{cases} u, & u < 2^{w-1} \\ u - 2^w, & u \geq 2^{w-1} \end{cases} \quad (2.8)$$

Mapping Signed $\leftrightarrow$ Unsigned

Bits	Signed		Unsigned
0000	0		0
0001	1		1
0010	2		2
0011	3		3
0100	4		4
0101	5	$\rightarrow$ T2U $\rightarrow$	5
0110	6		6
0111	7	$\leftarrow$ U2T $\leftarrow$	7
1000	-8		8
1001	-7		9
1010	-6	$= + 2^w$	10
1011	-5	$\rightarrow$	11
1100	-4		12
1101	-3	$- 2^w$	13
1110	-2	$\leftarrow$	14
1111	-1		15

Signed vs. Unsigned in C

★ Constants

- By default are considered to be signed integers
- Unsigned if have “U” as suffix

0U, 12345U, 0x1A2BU, 4294967259U

★ Casting

- Explicit casting between signed & unsigned same as U2T and T2U

```
int tx, ty;
```

```
unsigned ux, uy;
```

```
tx = (int) ux;
```

```
uy = (unsigned) ty;
```

- Implicit casting also occurs via assignments and procedure calls

```
tx = ux;
```

```
uy = ty;
```

Casting Surprises

* Expression Evaluation

- If there is a mix of unsigned and signed in single expression, ***signed values implicitly cast to unsigned***
- Including comparison operations $<$, $>$, $==$, $<=$, $>=$
- Examples for $W = 32$: ***TMIN = -2,147,483,648 , TMAX = 2,147,483,647***

* Constant ₁	Constant ₂	Relation	Evaluation
0 0	0U	==	unsigned
-1 -1	0	<	signed
-1 -1	0U	>	unsigned
2147483647	-2147483648	>	signed
2147483647	-2147483647-1	<	unsigned
2147483647U	-2147483647-1	>	signed
-1 -1	-2	>	unsigned
(unsigned)-1	-2	>	unsigned
2147483647	2147483648U	<	unsigned
2147483647	(int) 2147483648U	>	signed

Shift Operations

* **Left Shift:** $x \ll y$

- Shift bit-vector x left y positions
 - Throw away extra bits on left
- * Fill with 0's on right

* **Right Shift:** $x \gg y$

- Shift bit-vector x right y positions
 - * Throw away extra bits on right
- **Logical shift**
 - * Fill with 0's on left
- **Arithmetic shift**
 - * Replicate most significant bit on left

* **Undefined Behavior**

- Shift amount < 0 or $\geq$ word size

Argument x	01100010
$\ll 3$	00010000
Log. $\gg 2$	00011000
Arith. $\gg 2$	00011000

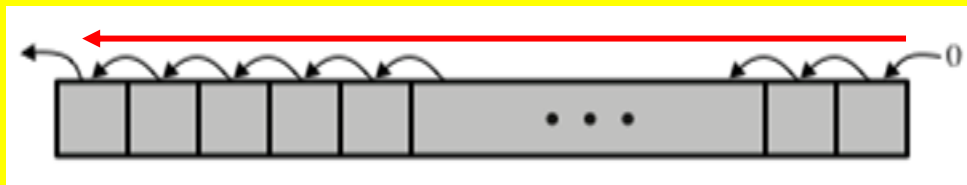
Argument x	10100010
$\ll 3$	00010000
Log. $\gg 2$	00101000
Arith. $\gg 2$	11101000

Shift Operations

❑ Given $x = 00001000 = 8$ $y = 11111000 = -8$

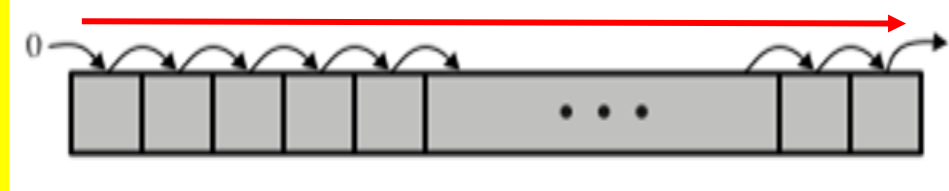
❖ Logical Shift Left

- * Bits are shifted left and 0 is pushed into LSB



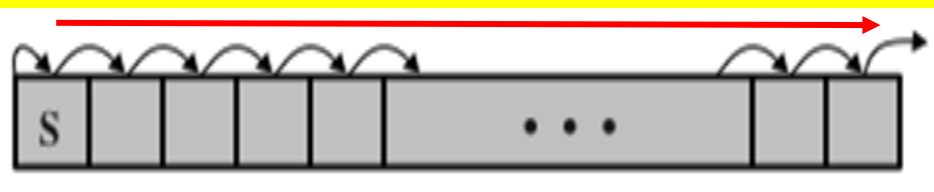
❖ Logical Shift Right

- * Bits are shifted right and 0 is pushed into MSB



* Arithmetic Shift Right

- * Bits are shifted right, MSB (Sign bit) is preserved and it is pushed in from MSB to the right



$x =$

0	0	0	0	1	0	0	0
---	---	---	---	---	---	---	---

Shift x logically left by 1 bit

16

0	0	0	1	0	0	0	0
---	---	---	---	---	---	---	---

Shift x logically right by 1 bit

4

0	0	0	0	0	1	0	0
---	---	---	---	---	---	---	---

$y =$

1	1	1	1	1	0	0	0
---	---	---	---	---	---	---	---

Shift y logically left by 1 bit

-16

1	1	1	1	0	0	0	0
---	---	---	---	---	---	---	---

Shift y logically right by 1 bit

0	1	1	1	1	1	0	0
---	---	---	---	---	---	---	---

Incorrect : Negative $\rightarrow$ Positive

Shift y arithmetically right by 1 bit

-4

1	1	1	1	1	1	0	0
---	---	---	---	---	---	---	---

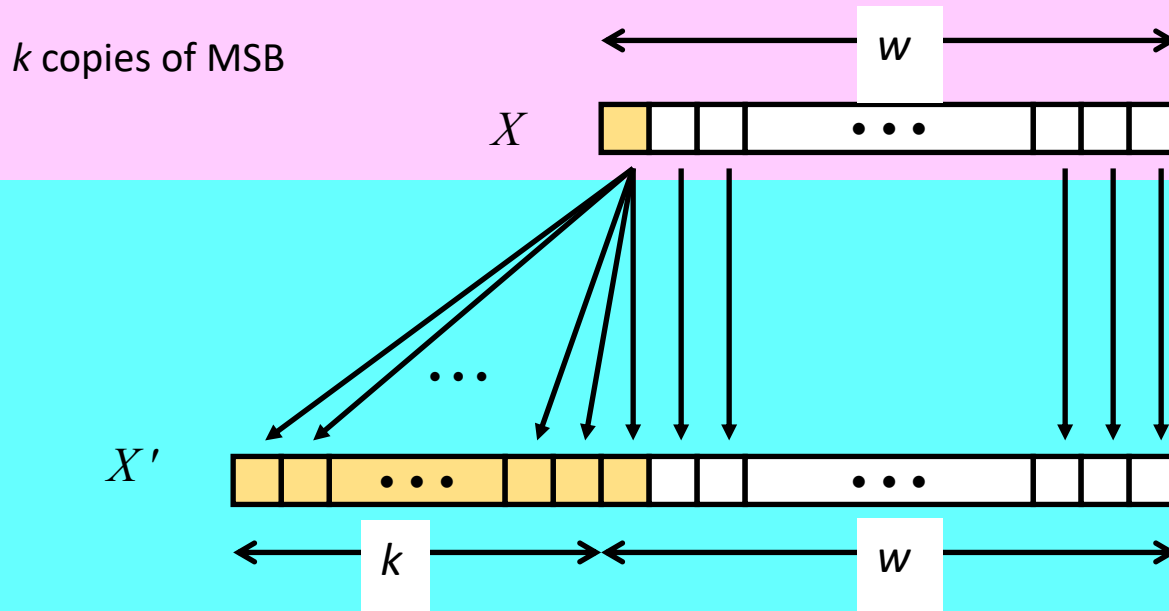
Sign Extension

★ Task:

- Given w -bit signed integer x
- Convert it to $w+k$ -bit integer with same value

★ Rule:

- Make k copies of sign bit:
- $X' = \underbrace{X_{w-1}, \dots, X_{w-1}}_{k \text{ copies of MSB}}, X_{w-1}, X_{w-2}, \dots, X_0$



Sign Extension Example

```
short int x = 15213;
int      ix = (int) x;
short int y = -15213;
int      iy = (int) y;
```

	Decimal	Hex	Binary
x	15213	3B 6D	00111011 01101101
ix	15213	00 00 3B 6D	00000000 00000000 00111011 01101101
y	-15213	C4 93	11000100 10010011
iy	-15213	FF FF C4 93	11111111 11111111 11000100 10010011

- ★ Converting from smaller to larger integer data type
- ★ C automatically performs sign extension

Truncation

- ★ For an integer number x , the result of truncating it to k bits is equivalent to computing $x \bmod 2^k$. This can be seen by applying the modulus operation. **This is known as Modular Arithmetic**

In computing, the modulo operation returns the remainder or signed remainder of a division, after one number is divided by another (called the *modulus* of the operation).

Given two positive numbers a and n , a modulo n (abbreviated as $a \bmod n$) is the remainder of the Euclidean division of a by n , where a is the dividend and n is the divisor.

The modulo operation is to be distinguished from the symbol mod, which refers to the modulus (or divisor) one is operating from.

For example : $125 \bmod 2 = 1$ $-90 \bmod 16 = -10$

the expression " $125 \bmod 2$ " would evaluate to 1, because 125 divided by 2 has a quotient of 64 and a remainder of 1,

while " $-90 \bmod 16$ " would evaluate to -10, because the division of -90 by 16 has a quotient of -5 and a remainder of -10

$y = 01010110 \bmod 2^4 \rightarrow$ Truncate to 4-bits $y = 0110$

$y = 0111011001101011 \bmod 2^8 \rightarrow$ Truncate to 8 bits $y = 01101011$

In C language mod operator is % - e.g., $a=125 \% 2$; $b=-90 \% 16$;

An application of Modular Arithmetic is in Cryptography

SOC 2040 SYSTEM PROGRAMMING

Reading Assignments

Chapter 2 of Text Book

➤ Computer Systems : A Programmer's Perspective

- Randal E. Bryant and David R. O'Hallaron
3rd Edition, Global Edition Prentice Hall 2016
ISBN 10:1-292-10176-8**

TRY OUT ALL PRACTICE PROBLEMS

★ CHAPTER 2 - Practice Problems



2.17(page 61 Solution page 139)

2.18(page 64 Solution page 139)

2.19(page 67 Solution page 140)

★ 2.20(page 68 Solution page 140)

★ 2.21(page 70 Solution page 140)

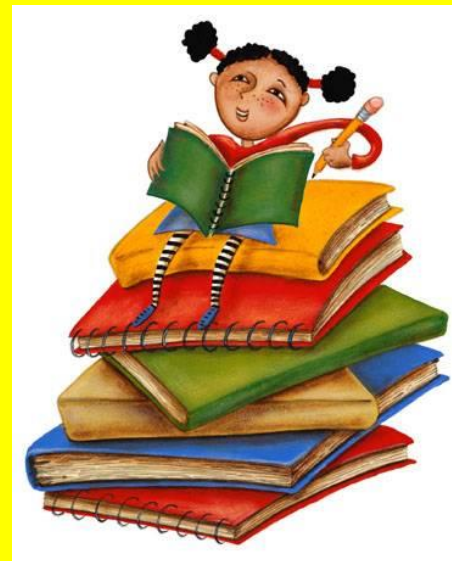
★ 2.22(page 74 Solution page 141)

★ 2.23(page 74 Solution page 141)

2.24(page 76 Solution page 141)

2.25(page 77 Solution page 142)

2.26(page 77 Solution page 142)



SOC 2040

SYSTEM PROGRAMMING

END OF
WEEK 4 LECTURE1

DATA REPRESENTATION

WISH YOU ALL THE BEST

INHA UNIVERSITY TASHKENT

SPRING SEMESTER 2024

SOC 2040

SYSTEM PROGRAMMING

CREDITS/HOURS PER WEEK : 3/3

COURSE TYPE : TECHNICAL CORE SEQUENCE

INSTRUCTOR

DR. A. R. NASEER

HEAD & PROFESSOR OF COMPUTER SCIENCE & ENGG

